

Full Documentation: Quadratic Curve Fitting

Senior Data Analyst & ML Specialist

December 3, 2025

1 Introduction

This document details the complete pipeline for analyzing a dataset, from importing raw Excel data to implementing a **Least Squares Regression** model manually. The objective is to model the relationship $y = f(x)$ where $f(x)$ is a quadratic function.

2 Phase 1: Data Import & Pre-processing

2.1 Loading Data from Source

The Code:

```
1 data_imported = readtable("MyData.xlsx");
2 data_array = table2array(data_imported);
```

Technical Explanation: The data is ingested from a structured file format ('.xlsx') into a MATLAB Table object. It is then converted into a numerical matrix $\mathbf{D} \in \mathbb{R}^{n \times m}$ (where $n = 100$ rows, $m = 2$ columns) to facilitate linear algebra operations.

2.2 Variable Extraction

The Code:

```
1 x = data_array(:,1);
2 y = data_array(:,2);
```

Mathematical Explanation: We decompose the matrix $\mathbf{D}$ into two column vectors:

- **Independent Variable (x):** The feature vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$.
- **Dependent Variable (y):** The target vector $\mathbf{y} = [y_1, y_2, \dots, y_n]^T$.

2.3 Exploratory Data Analysis (Visualization)

The Code:

```
1 figure(1);
2 plot(x, y, 'b-', 'LineWidth', 0.5);
3 grid on;
4 xlabel('independent variable');
5 ylabel('dependent variable');
6 title('Raw Data Plot');
```

Explanation: Before modeling, we perform visual inspection to verify the trend. The plot suggests a non-linear relationship (parabolic), justifying the choice of a 2^{nd} degree polynomial model ($y \propto x^2$).

3 Phase 2: The Least Squares Implementation

3.1 Constructing the Design Matrix (A)

The Code:

```
1 A = [ones(length(x), 1), x(:), x(:).^2];
```

Mathematical Explanation: We construct the *Vandermonde Design Matrix*. The system of linear equations is formulated as $\mathbf{A}\mathbf{c} = \mathbf{y}$:

$$\mathbf{A} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix}$$

3.2 Normal Equations Derivation

The Code:

```
1 A_Transpose = A.';
2 normal_matrix = A_Transpose * A;
3 rhs_vector = A_Transpose * y;
```

Mathematical Explanation: To solve for $\mathbf{c}$ when $\mathbf{A}$ is not square, we multiply by $\mathbf{A}^T$ to form the *Normal Equation*:

$$(\mathbf{A}^T \mathbf{A})\mathbf{c} = \mathbf{A}^T \mathbf{y}$$

Here, $(\mathbf{A}^T \mathbf{A})$ is a symmetric 3×3 matrix representing the curvature of the loss function.

3.3 Solving for Coefficients

The Code:

```
1 c = inv(normal_matrix) * rhs_vector;
```

Mathematical Explanation: We compute the optimal coefficient vector $\mathbf{c} = [c_{\text{intercept}}, c_{\text{slope}}, c_{\text{curvature}}]$ by inverting the normal matrix:

$$\mathbf{c} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y} \quad (1)$$

4 Phase 3: Model Evaluation

4.1 Validation against Built-in Functions

The Code:

```
1 poly_fit_coeff = flip(polyfit(x,y, 2)).';
```

Note: We use `flip` because MATLAB's `polyfit` returns descending powers ($ax^2 + bx + c$), while our manual derivation uses ascending powers ($c + bx + ax^2$).

4.2 Residual Analysis

The Code:

```
1 y_fitted = c(1) + c(2)*x + c(3)*(x.^2);
2 residual_evaluated = y - y_fitted;
3 norm_of_residuals = norm(residual_evaluated);
```

Mathematical Explanation: We calculate the *L2-Norm* of the error vector $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$:

$$\|\mathbf{e}\|_2 = \sqrt{\sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Minimizing this value was the primary objective of the algorithm.